

4장 딥러닝 시작

4.1 인공 신경망의 한계와 딥러닝 출현

퍼셉트론

오늘날 신경망(딥러닝)의 기원이 되는 알고리즘

다수의 신호를 입력으로 받아 하나의 신호를 출력, 신호를 입력으로 받아 1 또는 0 정보를 앞으로 전달하는 원리로 작동

AND 게이트 : 모든 입력이 '1'일 때 작동

OR 게이트 : 입력에서 둘 중 하나만 '1'이거나 둘 다 '1'일 때 작동 (1이 하나 이상)

XOR 게이트 : 두 개 중 한 개만 '1'일 때만 작동 (배타적 논리합)

AND, OR 게이트와 다르게 XOR 게이트는 데이터가 비선형적으로 분리됨

→ 단층 퍼셉트론에서 학습이 불가능함

→ 해결하기 위해 입력층과 출력층 사이에 하나 이상의 중간층 (은닉층)을 두어 비선형적으로 분리되는 데이터에 대해서도 학습이 가능하게함 = 다층 퍼셉트론

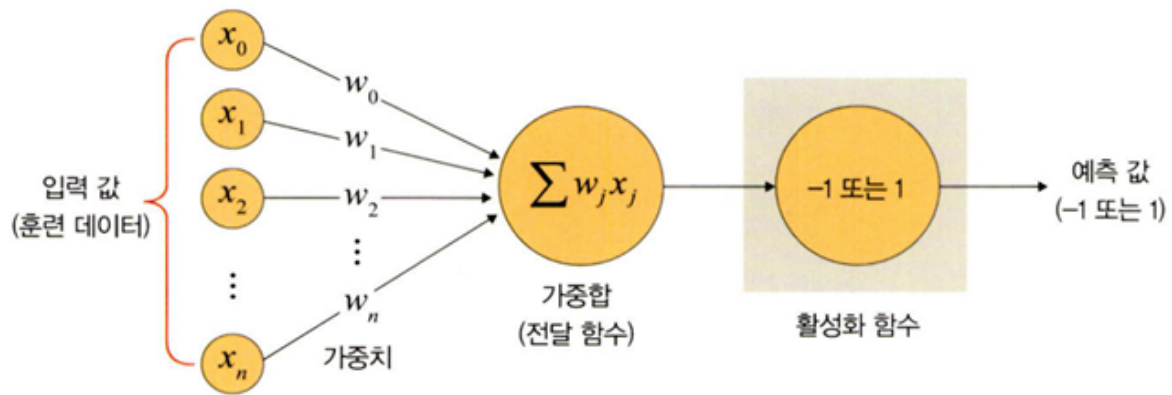
심층 신경망 (Deep Neural Network) : 입력층과 출력층 사이에 은닉층이 여러 개 있는 신경망 (= 딥러닝)

4.2 딥러닝 구조

딥러닝 용어

	구성요소	설명
층	입력층	데이터를 받아들이는 층
	은닉층	모든 입력 노드부터 입력값을 받아 가중합을 계산하고 이 값을 활성화 함수에 적용하여 출력층에 전달하는 층
	출력층	신경망의 최종 결과값이 포함된 층
가중치		노드와 노드 간 연결 강도
바이어스 (bias)		가중합에 더해 주는 상수로 하나의 뉴런에서 활성화 함수를 거쳐 최종적으로 출력되는 값을 조절하는 역할을 함
가중합 (전달 함수)		가중치와 신호의 곱을 합한 것

함수	활성화 함수	신호를 입력 받아 이를 적절히 처리하여 출력 해주는 함수
	손실 함수	가중치 학습을 위해 출력 함수의 결과와 실제 값 간의 오차를 측정하는 함수



가중치

입력 값이 연산 결과에 미치는 영향력을 조절하는 요소

입력 값이 아무리 커도 입력 값이 해당하는 가중치가 매우 작으면 (입력값 x 가중치) 값은 0 이거나 0에 가까운 값을 가지게 됨

가중합 (전달 함수)

각 노드에서 들어오는 신호에 가중치를 곱해서 다음 노드로 전달되는데, 이 값들을 모두 더한 합계를 가중합이라고 함

노드의 가중합이 계산되면 이 가중합을 활성화 함수로 보내기 때문에 전달 함수라고도 함

가중합 공식

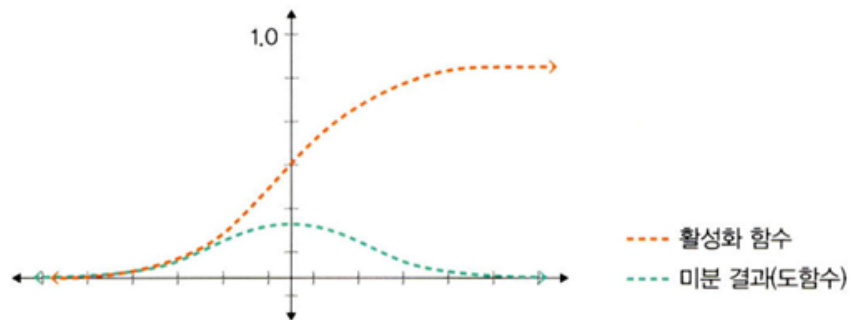
$$\sum_i w_i x_i + b \quad (w : \text{가중치}, b : \text{바이어스})$$

활성화 함수

전달 함수에서 전달 받은 값을 출력할 때 일정 기준에 따라 출력 값을 변화시키는 비선형 함수

sigmoid, tanh, ReLU 함수가 있음

- Sigmoid (시그모이드 함수)



$$f(x) = \frac{1}{1+e^{-x}}$$

선형 함수의 결과를 0~1 사이에서 비선형 형태로 변형

주로 로지스틱 회귀와 같은 분류 문제를 확률적으로 표현하는데 사용됨

하지만 모델의 깊이가 깊어지면 기울기가 사라지는 기울기 소멸 문제가 발생
때문에 딥러닝 모델에서는 잘 사용하지 않음

- tanh (하이퍼볼릭 탄젠트 함수)

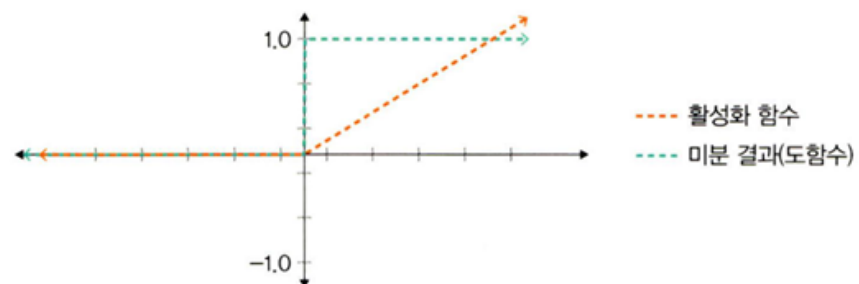


선형 함수의 결과를 -1~1 사이에서 비선형 형태로 변형

sigmoid에서는 양수 결과만 나와 결과값의 평균이 0이 아닌 양수로 편향된 문제가 있었지만 tanh에서는 이를 해결

하지만 여전히 기울기가 사라지는 기울기 소멸 문제가 발생

- ReLU



입력이 음수일 때는 0을 출력하고 양수일 때는 x를 출력

경사 하강법에 영향을 주지 않아 학습 속도가 빠름

기울기 소멸 문제가 발생하지 않음

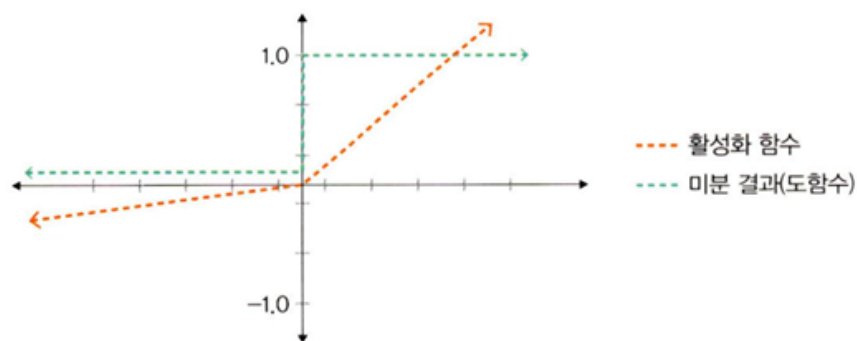
주로 은닉층에서 사용되며 tanh 함수에 비해 학습 속도가 6배 빠름

음수 값을 입력 받으면 항상 0을 출력하기 때문에 학습 능력이 감소

*결과 값이 0이면 미분 값도 0이 됨 → 뉴런의 가중치가 업데이트 되지 못함 → 해당 뉴런이 학습에 기여하지 못하게 됨 → 학습 능력 감소

⇒ Leaky ReLU

- Leaky ReLU



입력 값이 음수이면 0이 아닌 매우 작은 수를 반환

그러면 입력 값이 수렴하는 구간이 제거되어 ReLU 함수의 단점을 해결할 수 있음

- softmax (소프트 맥스 함수)

$$y_k = \frac{\exp(a_k)}{\sum_{i=1}^n \exp(a_i)} \quad (n \text{은 출력층의 뉴런 개수, } y_k \text{는 그중 } k \text{번째 출력을 의미})$$

입력 값을 0~1 사이에 출력되도록 정규화하여 출력 값들의 총합이 항상 1이 되도록 함
보통 출력 노드의 활성화 함수로 많이 사용됨

손실 함수

경사하강법

: 학습률과 손실 함수의 순간 기울기를 이용하여 가중치를 업데이트 하는 방법

: 미분의 기울기를 이용하여 오차를 비교하고 최소화하는 방향으로 이동시키는 방법

*학습률 : 한 번 학습할 때 얼마큼 변화를 주는지에 대한 상수

이때 손실 함수는 학습을 통해 얻은 데이터의 추정치가 실제 데이터와 얼마나 차이가 나는지 평가하는 지표

값이 클수록 많이 틀렸다는 의미, 0에 가까울수록 완벽하게 추정했다는 의미

- 평균 제곱 오차 (MSE)

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$\hat{y}_i$: 신경망의 출력(신경망이 추정한 값)
 y_i : 정답 레이블
 i : 데이터의 차원 개수

실제 값과 예측 값의 차이를 제공하여 평균을 낸 것
회귀에서 주로 사용

- 크로스 엔트로피 오차 (CrossEntropy)

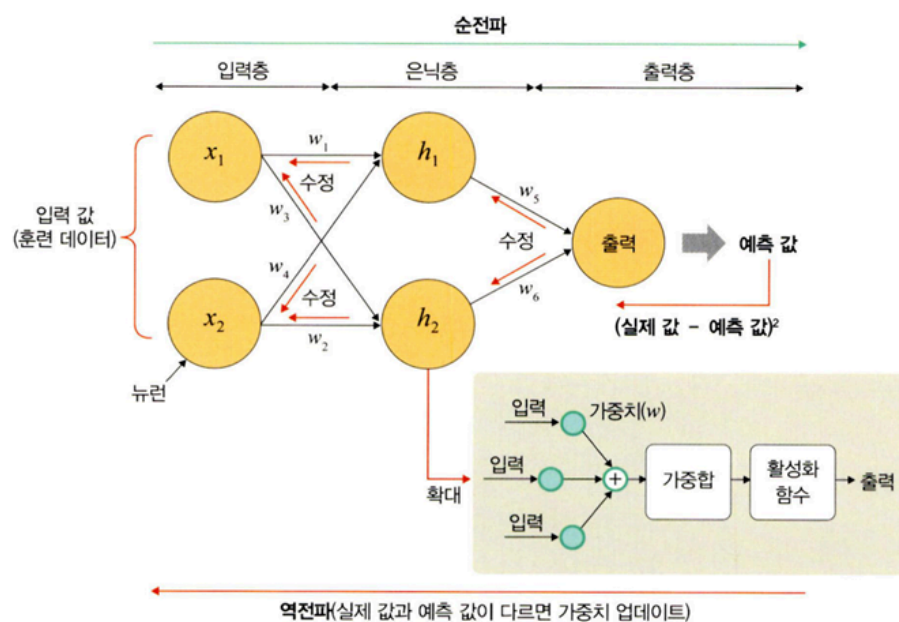
$$CrossEntropy = - \sum_{i=1}^n y_i \log \hat{y}_i$$

$\hat{y}_i$: 신경망의 출력(신경망이 추정한 값)
 y_i : 정답 레이블
 i : 데이터의 차원 개수

분류 문제에서 one-hot encoding을 했을 때만 사용할 수 있는 오차 계산법
보통 분류 문제에서 0과 1로 출력하기 위해 sigmoid 함수를 사용하는데, sigmoid 함수에 포함된 자연 상수 e 때문에 평균 제곱 오차를 적용하면 울퉁불퉁한 그래프가 출력됨

CrossEntropy를 적용할 경우 경사 하강법 과정에서 학습이 지역 최소점에서 멈출 수 있음
이를 방지하고자 자연 상수 e에 반대되는 자연 로그를 모델의 출력 값에 취함

딥러닝 학습



딥러닝 학습은 순전파와 역전파로 나뉨

- 순전파

네트워크에 훈련 데이터가 들어올때 발생

데이터를 기반으로 예측 값을 계산하기 위해 전체 신경망을 교차해 지나감

모든 뉴런이 이전 층의 뉴런에서 수신한 정보에 변환을 적용하여 다음 층의 뉴런으로 전송하는 방식

데이터가 모든 층을 통과하고 모든 뉴런이 계산을 완료하면 그 예측 값은 최종 층에 도달

예측 값이 나오게 되면 손실함수로 네트워크의 예측 값과 실제 값의 차이를 추정

손실 함수 비용이 0이면 이상적

- 역전파

손실이 계산되면 그 정보는 역으로 전파됨 (출력층 → 은닉층 → 입력층)

손실 함수 비용이 0에 가깝도록 하기 위해 모델이 훈련을 반복하면서 가중치를 조정

출력층에서 시작된 손실 비용은 은닉층의 모든 뉴런으로 전파되지만, 은닉층의 뉴런은 각 뉴런이 원래 출력에 기여한 상대적 기여도에 따라, 가중치에 따라, 값이 달라짐

이 과정을 모든 뉴런에 대해 진행하여 계산된 각 뉴런 결과를 또다시 순전파의 가중치 값으로 사용

딥러닝의 문제점과 해결 방안

딥러닝에서는 활성화 함수가 적용된 여러 은닉층을 결합하여 비선형 영역을 표현

은닉층 개수가 많을수록 데이터 분류가 잘되지만 문제점이 나타남

- 과적합 문제 발생

훈련 데이터를 과하게 학습한 것이 원인

훈련 데이터를 과하게 학습하면 예측 값과 실제 값 차이인 오차가 감소하지만 검증 데이터에 대해서는 오차가 증가함

⇒ Dropout (드롭아웃)

신경망 모델이 과적합 되는 것을 피하기 위한 방법

학습 과정 중 임의로 일부 노드들을 학습에서 제외시킴

- 기울기 소멸 문제 발생

출력층에서 은닉층으로 전달되는 오차가 크게 줄어들어 학습이 되지 않는 현상

기울기가 소멸되기 때문에 학습되는 양이 0에 가까워져 학습이 더디게 진행된다 오차를 더 줄이지 못하고 그 상태로 수렴하는 현상

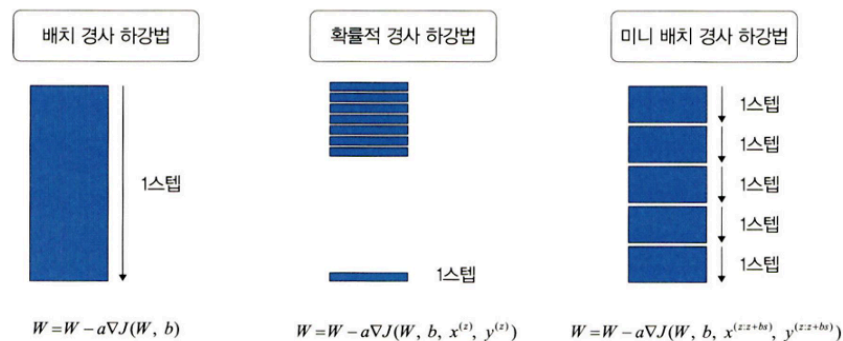
⇒ Sigmoid, tanh 대신 ReLU 함수 사용

- 성능이 나빠지는 문제 발생

경사하강법은 손실 함수의 비용이 최소가 되는 지점을 찾을 때까지 기울기가 낮은 쪽으로 계속 이동시키는 과정을 반복 → 이때 성능이 나빠지는 문제가 발생

⇒ 확률적 경사 하강법 or 미니 배치 경사 하강법 사용

경사하강법



- 배치 경사 하강법 (BGD)

전체 데이터셋에 대한 오류를 구한 후 기울기를 한 번만 계산하여 모델의 파라미터를 업데이트하는 방법

전체 훈련 데이터셋에 대해 가중치를 편미분하는 방법

한 스텝에 모든 훈련 데이터셋을 사용하므로 학습이 오래 걸림

⇒ 확률적 경사 하강법

- 확률적 경사 하강법 (SGD)

임의로 선택한 데이터에 대해 기울기를 계산하는 방법

적은 데이터를 사용하므로 빠른 계산이 가능

파라미터 변경 폭이 불안정하고, 때로는 배치 경사 하강법보다 정확도가 나중될 수 있지만 속도가 빠름

- 미니 배치 경사 하강법

전체 데이터셋을 미니 배치 여러개로 나누고, 미니 배치 한 개마다 기울기를 구한 후 그것의 평균 기울기를 이용하여 모델을 업데이트해서 학습하는 방법

전체 데이터를 계산하는 것보다 빠름

확률적 경사 하강법보다 안정적이어서 실제로 많이 사용

옵티마이저

확률적 경사 하강법의 파라미터 변경 폭이 불안정한 문제를 해결하기 위해 학습 속도와 운동량을 조정하는 옵티마이저 적용

속도 조정

- Adagrad (아다그라드)

가중치의 업데이트 횟수에 따라 학습률을 조정하는 방법

많이 변화하지 않는 변수들의 학습률은 크게 하고, 많이 변화하는 변수들의 학습률은 작게 함

→ 많이 변화한 변수는 최적 값에 근접했을 것이라는 가정하에 작은 크기로 이동하면서 세밀하게 값을 조정, 반대로 적게 변화한 변수들은 학습률을 크게 하여 빠르게 오차 값을 줄이고자 하는 방법

기울기가 0에 수렴하는 문제가 있어 사용하지 않음

⇒ RMSProp 사용

- Adadelta (아다델타)

Adagrad에서 기울기 크기의 누적 값이 커짐에 따라 학습이 멈추는 문제를 해결하기 위해 등장한 방법

Adagrad의 수식에서 학습률을 가중치의 변화량 크기를 누적한 값으로 변환했기 때문에 학습률에 대한 하이퍼파라미터가 필요하지 않음

- RMSProp

Adagrad의 기울기 크기의 누적 값이 무한히 커지는 것을 방지하고자 제안된 방법

값이 너무 크면 학습률이 작아져 학습이 안될 수 있으므로 사용자가 감마값을 이용하여 학습률 크기를 비율로 조정할 수 있도록 함

운동량 조정

- Momentum

매번 기울기를 구하지만 가중치를 수정하기 전에 이전 수정 방향을 참고하여 같은 방향으로 일정한 비율만 수정하는 방법

지그재그 현상이 줄어들고 이전 이동 값을 고려하여 일정 비율만큼 다음 값을 결정하므로 관성 효과를 얻을 수 있음

- NAG (네스테로프 모멘텀)

모멘텀 값과 기울기 값이 더해져 실제 값을 만드는 기존 모멘텀과 달리 모멘텀 값이 적용된 지점에서 기울기 값을 계산

멈추어야 할 시점에서도 관성에 의해 훨씬 멀리 갈 수 있는 모멘텀의 단점을 극복

모멘텀의 장점인 빠른 이동 속도를 유지하면서 멈추어야 할 적절한 시점에서 제동을 거는데 훨씬 용이함

속도와 운동량에 대한 혼용

- Adam (아담)

모멘텀과 RMSProp의 장점을 결합한 경사 하강법

딥러닝을 사용할 때 이점

- 특성 추출

머신러닝 알고리즘에서의 특성 추출은 매우 복잡하며 수집된 데이터에 대한 전문 지식이 필요했음

하지만 딥러닝에서는 이러한 특성 추출 과정을 알고리즘에 통합시킴

데이터 특성을 잘 잡아내고자 은닉층을 깊게 쌓는 방식으로 파라미터를 늘린 모델 구조 덕분

- 빅데이터의 효율적 활용

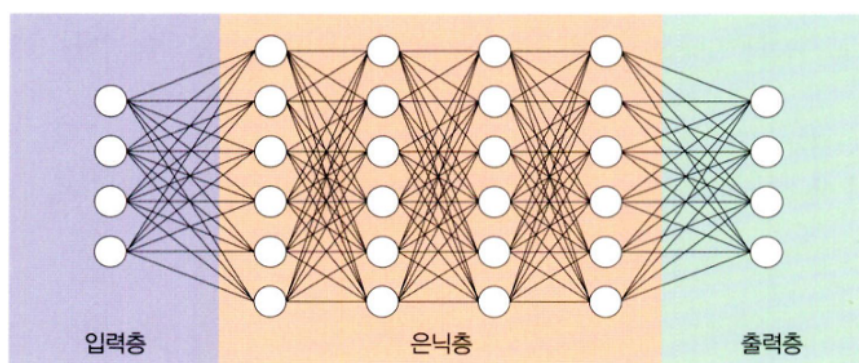
딥러닝에서 특성 추출을 알고리즘에 통합시켰는데 이것이 가능한 이유는 빅데이터 때문
딥러닝 학습을 이용한 특성 추출은 데이터 사례가 많을수록 성능이 향상됨

4.3 딥러닝 알고리즘

딥러닝 알고리즘은 심층 신경망을 사용한다는 공통점이 있음

머신러닝 알고리즘처럼 목적에 따라 나뉨

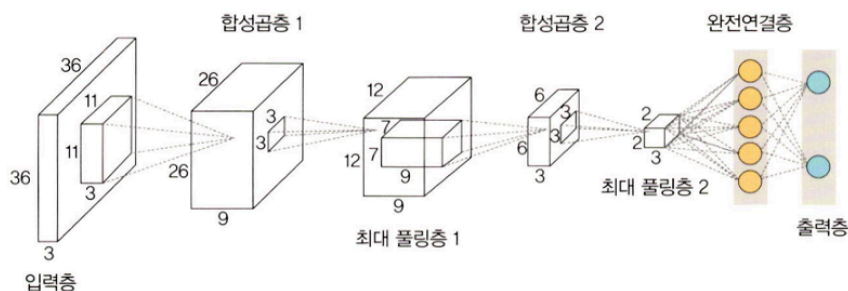
심층 신경망 (DNN)



입력층과 출력층 사이에 다수의 은닉층을 포함하는 인공 신경망

다수의 은닉층을 두었기 때문에 다양한 비선형 관계를 학습할 수 있음
 하지만 학습을 위한 연산량이 많고 기울기 소멸 문제 등이 발생할 수 있음
 이를 해결하기 위해 dropout, ReLU 함수, 배치 정규화 등을 적용해야 함

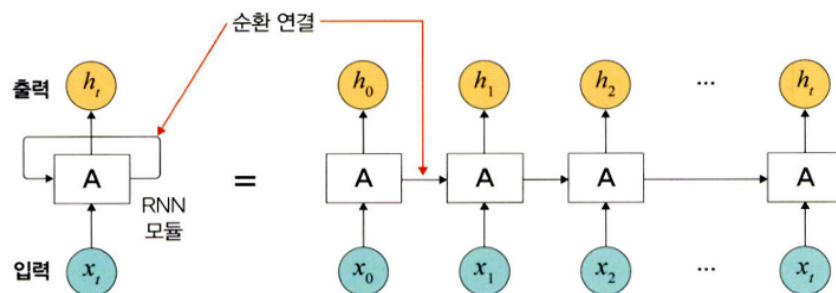
합성곱 신경망 (CNN)



합성곱층과 풀링층을 포함하는 이미지 처리 성능이 좋은 인공 신경망 알고리즘
 영상 및 사진이 포함된 이미지 데이터에서 객체를 탐색하거나 객체 위치를 찾아내는데 유용
 기존 신경망과의 차별점이 있음

- 각 층의 입출력 형상을 유지
- 이미지의 공간 정보를 유지하면서 인접 이미지와 차이가 있는 특징을 효과적으로 인식
- 복수 필터로 이미지의 특징을 추출하고 학습
- 추출한 이미지의 특징을 모으고 강화하는 풀링층이 있음
- 필터를 공유 파라미터로 사용하기 때문에 일반 인공 신경망과 비교하여 학습 파라미터가 매우 적음

순환 신경망 (RNN)



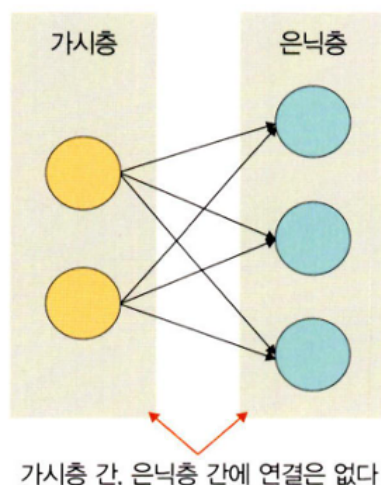
시계열 데이터 같은 시간 흐름에 따라 변화하는 데이터를 학습하기 위한 인공 신경망
 순환은 자기 자신을 참조한다는 것으로 현재 결과가 이전 결과와 연관이 있다는 의미

순환 신경망의 특징

- 시간성을 가진 데이터가 많음
- 시간정 정보를 이용하여 데이터의 특징을 잘 다룸

- 시간에 따라 내용이 변하므로 데이터는 동적이고, 길이가 가변적임
 - 매우 긴 데이터를 처리하는 연구가 활발히 진행되고 있음
- 기울기 소멸 문제로 학습이 제대로 되지 않는 경우가 있음
 ⇒ 메모리 개념을 도입한 LSTM이 사용됨

제한된 볼츠만 머신 (RBM)

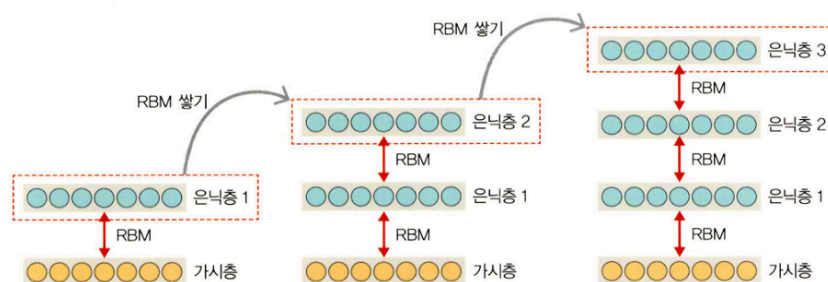


가시층과 은닉층으로 구성된 모델
 가시층은 은닉층과만 연결됨 (가시층끼리, 은닉층끼리는 연결이 없음)

제한된 볼츠만 머신의 특징

- 차원 감소, 분류, 선형 회귀 분석, 협업 필터링, 특성 값 학습, 주제 모델링에 사용
- 기울기 소멸 문제를 해결하기 위해 사전 학습 용도로 활용 가능
- 심층 신뢰 신경망의 요소로 활용됨

심층 신뢰 신경망 (DBN)



입력층과 은닉층으로 구성된 제한된 볼츠만 머신을 블록처럼 여러 층으로 쌓은 형태로 연결된 신경망

사전 훈련된 제한된 볼츠만 머신을 층층이 쌓아 올린 구조로, 레이블이 없는 데이터에 대한

비지도 학습이 가능함

부분적인 이미지에서 전체를 연상하는 일반화와 추상화 과정을 구현할 때 사용하면 유용

심층 신뢰 신경망의 특징

- 순차적으로 심층 신뢰 신경망을 학습시켜 가면서 계층적 구조를 생성
- 비지도 학습으로 학습
- 위로 올라갈수록 추상적 특성을 추출
- 학습된 가중치를 다층 퍼셉트론의 가중치 초기값으로 사용

4.4 우리는 무엇을 배워야 할까?

간단한 선형 회귀 분류를 이용하여 원하는 값을 도출할 수 있다면 머신 러닝만으로도 충분하지만 복잡한 비선형 데이터에 대한 분류 및 예측을 도출하고 싶다면 딥러닝으로 학습해야함

머신 러닝과 딥러닝은 얻고 싶은 결과를 도출하기 위한 도구일 뿐